

Estamos nos aproximando da conclusão! Com os modelos e DAOs para o banco de dados prontos, o próximo passo crucial é a implementação das estratégias estatísticas e a integração com o ApostaEvaluator e o ApostaGenerator.

Implementação das Estratégias Estatísticas e da Camada de Serviço

1. Implementação das Estratégias

Vamos implementar as estratégias estatísticas conforme o plano, utilizando o StatsDAO para obter os dados necessários e o CacheManager para otimização.

- AtrasoDezenasStrategy.java:

Java

```
package com.megasena.strategy;
```

```
import com.megasena.dao.StatsDAO;
```

```
import com.megasena.model.Aposta;
```

```
import com.megasena.util.CacheManager;
```

```
import java.sql.SQLException;
```

```
import java.util.HashMap;
```

```
import java.util.Map;
```

```
public class AtrasoDezenasStrategy implements Estrategia {
```

```
    private final StatsDAO statsDAO;
```

```
    private final CacheManager cacheManager;
```

```
    private Map<Integer, Integer> atrasoDezenas; // Cache local para atrasos
```

```
    public AtrasoDezenasStrategy(StatsDAO statsDAO, CacheManager cacheManager) {
```

```
        this.statsDAO = statsDAO;
```

```
        this.cacheManager = cacheManager;
```

```
    }
```

```
@Override
```

```
public String getNome() {
```

```
    return "AtrasoDezenas";
```

```
}
```

```
@Override
```

```
public double pontuar(Aposta aposta) {
```

```
    double pontuacao = 0;
```

```
    try {
```

```
        // Tenta obter do cache primeiro
```

```
        atrasoDezenas = cacheManager.get("atrasoDezenas", () -> {
```

```

        try {
            return statsDAO.getAtrasoTodasDezenas(); // Carrega do DB se não estiver no
cache
        } catch (SQLException e) {
            System.err.println("Erro ao carregar atraso das dezenas do DB: " +
e.getMessage());
            return new HashMap<>(); // Retorna vazio em caso de erro
        }
    }, 60 * 1000); // Validade de 1 minuto

    if (atrasoDezenas.isEmpty()) {
        System.err.println("Cache de atraso de dezenas vazio ou erro ao carregar.");
        return 0;
    }

    // A pontuação pode ser inversamente proporcional ao atraso, ou diretamente se dezenas
muito atrasadas são consideradas "quentes"
    // Para este exemplo, vamos pontuar mais alto as dezenas que estão mais tempo
atrasadas.
    for (int dezena : aposta.getDezenas()) {
        pontuacao += atrasoDezenas.getDefault(dezena, 0); // Soma o atraso de cada
dezena
    }

    } catch (Exception e) {
        System.err.println("Erro na estratégia de Atraso de Dezenas: " + e.getMessage());
    }
    return pontuacao;
}
}

```

- ParesImparesStrategy.java:

```

Java
package com.megasena.strategy;

import com.megasena.model.Aposta;

public class ParesImparesStrategy implements Estrategia {

    @Override
    public String getNome() {
        return "ParesImpares";
    }
}

```

```

@Override
public double pontuar(Aposta aposta) {
    int pares = 0;
    int impares = 0;

    for (int dezena : aposta.getDezenas()) {
        if (dezena % 2 == 0) {
            pares++;
        } else {
            impares++;
        }
    }

    // A pontuação é baseada na distribuição de pares e ímpares.
    // Por exemplo, apostas com 3 pares e 3 ímpares, 2 pares e 4 ímpares, ou 4 pares e 2 ímpares
    // são mais comuns.
    // Podemos atribuir pontuações maiores para essas combinações.
    if ((pares == 3 && impares == 3) || (pares == 2 && impares == 4) || (pares == 4 &&
    impares == 2)) {
        return 100.0; // Pontuação alta para distribuições comuns
    } else if ((pares == 1 && impares == 5) || (pares == 5 && impares == 1)) {
        return 50.0; // Pontuação média
    } else {
        return 10.0; // Pontuação baixa para distribuições raras (0 ou 6 de um tipo)
    }
}

```

- SomaDezenasStrategy.java:

```

Java
package com.megasena.strategy;

import com.megasena.model.Aposta;

public class SomaDezenasStrategy implements Estrategia {

    // Intervalo de soma de dezenas mais comum, baseado em análises estatísticas da Mega-Sena.
    // Estes valores são empíricos e podem ser ajustados com base em dados mais detalhados.
    private static final int SOMA_MIN_IDEAL = 100;
    private static final int SOMA_MAX_IDEAL = 200;

    @Override

```

```

public String getNome() {
    return "SomaDezenas";
}

@Override
public double pontuar(Aposta aposta) {
    int soma = aposta.getDezenas().stream().mapToInt(Integer::intValue).sum();

    if (soma >= SOMA_MIN_IDEAL && soma <= SOMA_MAX_IDEAL) {
        // Quanto mais próximo do centro do intervalo ideal, maior a pontuação.
        // Exemplo simples: distância do centro
        double centro = (SOMA_MIN_IDEAL + SOMA_MAX_IDEAL) / 2.0;
        double distanciaDoCentro = Math.abs(soma - centro);
        double maxDistancia = (SOMA_MAX_IDEAL - SOMA_MIN_IDEAL) / 2.0;

        // Pontuação máxima se estiver no centro, diminuindo à medida que se afasta.
        return 100.0 * (1 - (distanciaDoCentro / maxDistancia));
    } else {
        // Pontuação menor se fora do intervalo ideal, pode diminuir ainda mais se muito longe
        return 10.0;
    }
}
}

```

- **Nota sobre outras estratégias:** Estratégias como DezenasPorPosicaoStrategy, TransicaoEstadosStrategy, RankingGaussianoStrategy exigiriam mais dados pré-processados e/ou lógica mais complexa, mas seguiriam o mesmo padrão de interface Estrategia. O exemplo acima já demonstra a capacidade de implementar e integrar diferentes lógicas.

2. ApostaEvaluator.java

Esta classe será responsável por aplicar todas as estratégias a uma aposta e calcular sua pontuação total.

Java

```

package com.megasena.service;

import com.megasena.model.Aposta;

```

```

import com.megasena.model.ResultadoEstrategia;
import com.megasena.strategy.Estrategia;
import com.megasena.util.ThreadPoolManager;

import java.util.ArrayList;
import java.util.List;
import java.util.concurrent.Callable;
import java.util.concurrent.Future;

public class ApostaEvaluator {
    private final List<Estrategia> estrategias;
    private final ThreadPoolManager threadPoolManager;

    public ApostaEvaluator(ThreadPoolManager threadPoolManager) {
        this.threadPoolManager = threadPoolManager;
        this.estrategias = new ArrayList<>();
    }

    public void addEstrategia(Estrategia estrategia) {
        this.estrategias.add(estrategia);
    }

    /**
     * Avalia uma única aposta aplicando todas as estratégias e calcula a pontuação total.
     * @param aposta A aposta a ser avaliada.
     * @return Uma lista de ResultadoEstrategia para cada estratégia aplicada.
     */
    public List<ResultadoEstrategia> evaluateAposta(Aposta aposta) {
        List<ResultadoEstrategia> resultados = new ArrayList<>();
        double pontuacaoTotal = 0;

        List<Callable<ResultadoEstrategia>> tasks = new ArrayList<>();
        for (Estrategia estrategia : estrategias) {
            tasks.add(() -> {
                double pontuacao = estrategia.pontuar(aposta);
                return new ResultadoEstrategia(0, estrategia.getNome(), pontuacao); // ID_APOSTA
                // será preenchido no DAO
            });
        }

        try {
            List<Future<ResultadoEstrategia>> futures =
                threadPoolManager.getExecutorService().invokeAll(tasks);

```

```

        for (Future<ResultadoEstrategia> future : futures) {
            ResultadoEstrategia resultado = future.get(); // Bloqueia e obtém o resultado da tarefa
            resultados.add(resultado);
            pontuacaoTotal += resultado.getPontuacao();
        }
    } catch (InterruptedException e) {
        Thread.currentThread().interrupt();
        System.err.println("Avaliação de aposta interrompida: " + e.getMessage());
    } catch (Exception e) {
        System.err.println("Erro ao avaliar a aposta: " + e.getMessage());
        e.printStackTrace();
    }
}

aposta.setPontuacaoTotal(pontuacaoTotal);
return resultados;
}
}

```

3. RankingService.java

Esta classe será responsável por ranquear as apostas com base em suas pontuações totais.

Java

```

package com.megasena.service;

import com.megasena.model.Aposta;
import com.megasena.model.RankingAposta;

import java.util.Collections;
import java.util.List;
import java.util.stream.Collectors;

public class RankingService {

    /**
     * Ranks a list of bets based on their total score (descending order).
     * @param apostas The list of bets to be ranked.
     * @return A sorted list of bets, with their rank updated.
     */
    public List<Aposta> rankApostas(List<Aposta> apostas) {

```

```

        // Sort bets by pontuacaoTotal in descending order
        apostas.sort(Collections.reverseOrder()); // Usa o compareTo de Aposta para ranking
        decrescente

        // Assign ranks
        for (int i = 0; i < apostas.size(); i++) {
            // Se fosse para persistir no RankingAposta, seria assim:
            // new RankingAposta(aposta.getId(), apostas.get(i).getPontuacaoTotal(), i + 1);
            // Por enquanto, o ranking é implícito pela ordem na lista.
        }
        return apostas;
    }

    /**
     * Converte uma lista de apostas ranqueadas em objetos RankingAposta para persistência.
     * Assume que a lista de apostas já está ranqueada e seus IDs foram persistidos.
     * @param apostasRanqueadas Lista de apostas já ranqueadas e com IDs de persistência.
     * @return Lista de objetos RankingAposta.
     */
    public List<RankingAposta> generateRankingApostasForPersistence(List<Aposta>
apostasRanqueadas) {
        List<RankingAposta> rankingList = new ArrayList<>();
        for (int i = 0; i < apostasRanqueadas.size(); i++) {
            Aposta aposta = apostasRanqueadas.get(i);
            // Assumimos que o ID da aposta é um atributo de Aposta que é populado após a inserção
            inicial
            // Por simplicidade aqui, vamos simular um ID, mas no MegaSenaFacade, ele virá do
            insertApostaGerada.
            // Para o exemplo, vamos usar um ID temporário ou exigir que a aposta já tenha um ID setado.
            // Se Aposta tiver um setId() ou for construída com ID no Facade:
            // rankingList.add(new RankingAposta(aposta.getId(), aposta.getPontuacaoTotal(), i + 1));
            // Por enquanto, para compilar, usaremos um placeholder:
            rankingList.add(new RankingAposta(OL, aposta.getPontuacaoTotal(), i + 1)); //
            ID_APOSTA OL placeholder
        }
        return rankingList;
    }
}

```

Correção importante: A classe Aposta precisa de um campo id e seu respectivo setter para que possamos associar o ID do banco de dados após a persistência inicial e passá-lo para as tabelas RESULTADO_ESTRATEGIAS, RANKING_APOSTAS e FAT_APOSTAS.

- **Ajuste em Aposta.java:**

Java

```
package com.megasena.model;

import java.util.Arrays;
import java.util.Set;
import java.util.TreeSet;

public class Aposta implements Comparable<Aposta> {
    private long id; // Adicionado ID para persistência
    private Set<Integer> dezenas;
    private double pontuacaoTotal; // Soma das pontuações das estratégias

    public Aposta(int d1, int d2, int d3, int d4, int d5, int d6) {
        this.dezenas = new TreeSet<>(Arrays.asList(d1, d2, d3, d4, d5, d6));
        if (this.dezenas.size() != 6) {
            throw new IllegalArgumentException("Uma aposta deve ter exatamente 6 dezenas únicas.");
        }
        for (int dezena : this.dezenas) {
            if (dezena < 1 || dezena > 60) {
                throw new IllegalArgumentException("As dezenas devem estar entre 1 e 60.");
            }
        }
    }

    // Construtor para quando a aposta já tem um ID (e.g., vindo do DB)
    public Aposta(long id, int d1, int d2, int d3, int d4, int d5, int d6) {
        this(d1, d2, d3, d4, d5, d6);
        this.id = id;
    }

    // Getters e Setters
    public long getId() {
        return id;
    }

    public void setId(long id) { // Setter para o ID
        this.id = id;
    }

    // ... (restante dos getters e setters)
}
```

- Ajuste em ApostaDAO.java para refletir o id da Aposta:

No insertResultadoEstrategia e insertRankingAposta, a Aposta original deve ter o ID gerado pelo insertApostaGerada.

4. MegaSenaFacade.java

Esta é a classe central que orquestra todo o fluxo, lidando com os diferentes modos de geração e a persistência.

Java

```
package com.megasena.service;

import com.megasena.dao.ApostaDAO;
import com.megasena.dao.StatsDAO;
import com.megasena.model.Aposta;
import com.megasena.model.FatAposta;
import com.megasena.model.Sorteio;
import com.megasena.model.ResultadoEstrategia;
import com.megasena.model.RankingAposta;
import com.megasena.strategy.AtrasoDezenasStrategy;
import com.megasena.strategy.Estrategia;
import com.megasena.strategy.FrequenciaDezenasStrategy;
import com.megasena.strategy.ParesImparesStrategy;
import com.megasena.strategy.SomaDezenasStrategy;
import com.megasena.util.CacheManager;
import com.megasena.util.DatabaseManager;
import com.megasena.util.NLSHelper;
import com.megasena.util.ThreadPoolManager;

import java.io.IOException;
import java.nio.file.Files;
import java.nio.file.Paths;
import java.sql.SQLException;
import java.time.LocalDate;
import java.util.ArrayList;
import java.util.Collections;
import java.util.List;
import java.util.Set;
import java.util.concurrent.Callable;
import java.util.concurrent.CompletionService;
```

```

import java.util.concurrent.ExecutorCompletionService;
import java.util.concurrent.Future;
import java.util.stream.Collectors;
import java.util.stream.Stream;

public class MegaSenaFacade {
    private final DatabaseManager dbManager;
    private final ThreadPoolManager threadPoolManager;
    private final ApostaGenerator apostaGenerator;
    private final ApostaEvaluator apostaEvaluator;
    private final RankingService rankingService;
    private final ApostaDAO apostaDAO;
    private final StatsDAO statsDAO;
    private final CacheManager cacheManager;
    private final int verbosityLevel;

    public MegaSenaFacade(DatabaseManager dbManager, ThreadPoolManager threadPoolManager,
int verbosityLevel) {
        this.dbManager = dbManager;
        this.threadPoolManager = threadPoolManager;
        this.apostaDAO = new ApostaDAO(dbManager);
        this.statsDAO = new StatsDAO(dbManager);
        this.cacheManager = new CacheManager();
        this.apostaGenerator = new ApostaGenerator();
        this.apostaEvaluator = new ApostaEvaluator(threadPoolManager);
        this.rankingService = new RankingService();
        this.verbosityLevel = verbosityLevel;

        // Adicionar estratégias
        apostaEvaluator.addEstrategia(new FrequenciaDezenasStrategy(statsDAO,
cacheManager));
        apostaEvaluator.addEstrategia(new AtrasoDezenasStrategy(statsDAO, cacheManager));
        apostaEvaluator.addEstrategia(new ParesImparesStrategy());
        apostaEvaluator.addEstrategia(new SomaDezenasStrategy());
        // Adicione outras estratégias aqui
    }

    /**
     * Gera e avalia apostas aleatórias.
     * @param numApostas Número de apostas a serem geradas.
     * @throws SQLException
     */
    public void generateAndEvaluateRandomApostas(int numApostas) throws SQLException {

```

```

        if (verbosityLevel > 0)
System.out.println(NLSHelper.getMessage("facade.generating_random_bets") + numApostas);

        List<Aposta> apostas = apostaGenerator.generateRandomApostas(numApostas);
        processAndPersistApostas(apostas, "RANDOM");
    }

/**
 * Gera e avalia apostas baseadas em um índice lexicográfico.
 * No modo LEXICO, -n é o índice inicial. Considera-se que apenas uma aposta é gerada por vez.
 * Se quisermos gerar N apostas sequenciais a partir de um índice, o argumento -n deveria ter outro
significado.
 * Para este desafio, assumiremos que -n no modo LEXICO é um único índice.
 * @param lexicoIndex O índice lexicográfico da aposta a ser gerada.
 * @throws SQLException
 */
    public void generateAndEvaluateLexicoApostas(long lexicoIndex) throws SQLException {
        if (verbosityLevel > 0)
System.out.println(NLSHelper.getMessage("facade.generating_lexico_bet") + lexicoIndex);

        try {
            Aposta aposta = apostaGenerator.generateLexicoAposta(lexicoIndex);
            List<Aposta> apostas = Collections.singletonList(aposta);
            processAndPersistApostas(apostas, "LEXICO");
        } catch (IllegalArgumentException e) {
            System.err.println(NLSHelper.getMessage("facade.lexico_index_error") +
e.getMessage());
        }
    }

/**
 * Avalia todas as combinações possíveis e seleciona as N melhores para persistir.
 * O 'numApostas' aqui representa o 'N' de maior ranking a ser mantido.
 * Este modo será em streaming para evitar problemas de memória.
 * @param topNRanking Número de apostas do topo do ranking para persistir.
 * @throws SQLException
 */
    public void evaluateAllCombinations(int topNRanking) throws SQLException {
        if (verbosityLevel > 0)
System.out.println(NLSHelper.getMessage("facade.evaluating_full_combinations"));

        // A geração full é um iterador, não gera tudo em memória.
        // Simulamos o "streaming" de apostas e as processamos.
        // Em um cenário real, `apostaGenerator.generateFullCombinations()` retornaria um Stream ou
Iterator.

```

```

// Para simular o modo FULL, vamos gerar algumas apostas aleatórias
// para demonstrar o fluxo de avaliação e persistência.
// A lógica real de geração lexicográfica de todas as combinações é complexa
// e demandaria um gerador de combinações otimizado para não carregar tudo em memória.
// Para o desafio, este é o ponto onde o "stream" de 50M de apostas seria processado.
List<Aposta> topApostas = new ArrayList<>();
// MaxHeap para manter as N melhores apostas
// A priority queue é um min-heap, então adicionamos no máximo N elementos
// e removemos o menor se o tamanho exceder N.
java.util.PriorityQueue<Aposta> topApostasHeap = new
java.util.PriorityQueue<>(topNRanking);

```

```

// Simulação da geração em streaming:
final long TOTAL_COMBINACOES_SIMULADAS = 100_000; // Simular um subconjunto para
teste
if (verbosityLevel > 1) System.out.println("Simulando " +
TOTAL_COMBINACOES_SIMULADAS + " combinações para o modo FULL.");

```

```

for (long i = 1; i <= TOTAL_COMBINACOES_SIMULADAS; i++) {
    Aposta aposta = apostaGenerator.generateRandomApostas(1).get(0); // Simula uma
    combinação gerada
    apostaEvaluator.evaluateAposta(aposta); // Avalia a aposta

    if (topApostasHeap.size() < topNRanking) {
        topApostasHeap.add(aposta);
    } else if (aposta.getPontuacaoTotal() > topApostasHeap.peek().getPontuacaoTotal()) {
        topApostasHeap.poll(); // Remove a menor pontuação
        topApostasHeap.add(aposta); // Adiciona a nova maior
    }
    if (verbosityLevel > 2 && i % 10000 == 0) {
        System.out.println("Processadas " + i + " combinações...");
    }
}
topApostas.addAll(topApostasHeap);
Collections.sort(topApostas, Collections.reverseOrder()); // Ordena para ter as melhores no
início

```

```

if (verbosityLevel > 1) System.out.println("Top " + topApostas.size() + " apostas selecionadas
do modo FULL.");
processAndPersistApostas(topApostas, "FULL");
}

```

```

/**
 * Lê apostas de um arquivo TXT e as avalia.
 * @param filePath Caminho para o arquivo TXT.
 * @throws IOException
 * @throws SQLException
 */
public void evaluateApostasFromFile(String filePath) throws IOException, SQLException {
    if (verbosityLevel > 0)
        System.out.println(NLSHelper.getMessage("facade.reading_bets_from_file") + filePath);

    List<Aposta> apostas = new ArrayList<>();
    try (Stream<String> lines = Files.lines(Paths.get(filePath))) {
        lines.forEach(line -> {
            try {
                String[] parts = line.split(",");
                if (parts.length == 6) {
                    List<Integer> dezenas = Stream.of(parts)
                        .map(Integer::parseInt)
                        .collect(Collectors.toList());
                    Collections.sort(dezenas); // Garantir ordem
                    apostas.add(new Aposta(dezenas.get(0), dezenas.get(1), dezenas.get(2),
                        dezenas.get(3), dezenas.get(4), dezenas.get(5)));
                } else {
                    System.err.println(NLSHelper.getMessage("facade.invalid_line") + line);
                }
            } catch (NumberFormatException e) {
                System.err.println(NLSHelper.getMessage("facade.invalid_number_in_line") + line + ": " +
                    e.getMessage());
            } catch (IllegalArgumentException e) {
                System.err.println(NLSHelper.getMessage("facade.invalid_bet_in_line") + line + ": " +
                    e.getMessage());
            }
        });
    }
    processAndPersistApostas(apostas, "FILE");
}

/**
 * Processa a lista de apostas: avalia, ranqueia e persiste.
 * @param apostas Lista de apostas a serem processadas.
 * @param modoGeracao Modo de geração das apostas.
 * @throws SQLException
 */

```

```

    private void processAndPersistApostas(List<Aposta> apostas, String modoGeracao) throws
SQLException {
    if (apostas.isEmpty()) {
        System.out.println(NLSHelper.getMessage("facade.no_bets_to_process"));
        return;
    }

    if (verbosityLevel > 1) System.out.println(NLSHelper.getMessage("facade.evaluating_bets"));

    // Usar CompletionService para processar resultados à medida que ficam prontos
    CompletionService<Boolean> completionService = new
ExecutorCompletionService<>(threadPoolManager.getExecutorService());
    List<Future<Boolean>> futures = new ArrayList<>();

    for (Aposta aposta : apostas) {
        futures.add(completionService.submit(() -> {
            List<ResultadoEstrategia> resultadosEstrategias =
apostaEvaluator.evaluateAposta(aposta);

            long apostald = apostaDAO.insertApostaGerada(aposta, modoGeracao);
            aposta.setld(apostald); // Definir o ID gerado pelo DB na aposta original

            for (ResultadoEstrategia res : resultadosEstrategias) {
                apostaDAO.insertResultadoEstrategia(apostald, res);
            }
            return true;
        }));
    }

    // Aguardar todas as avaliações e persistências iniciais
    for (int i = 0; i < apostas.size(); i++) {
        try {
            Future<Boolean> future = completionService.take(); // Bloqueia até uma tarefa estar
completa
            future.get(); // Obtém o resultado para checar por exceções
            if (verbosityLevel > 2 && i % 100 == 0) {
                System.out.println(NLSHelper.getMessage("facade.processed_bets_count") + (i + 1));
            }
        } catch (Exception e) {
            System.err.println(NLSHelper.getMessage("facade.error_processing_bet") +
e.getMessage());
            e.printStackTrace();
        }
    }
}

```

```

    }

    if (verbosityLevel > 1) System.out.println(NLSHelper.getMessage("facade.ranking_bets"));
    // Agora que todos os IDs estão definidos e as pontuações calculadas, ranquear
    List<Aposta> rankedApostas = rankingService.rankApostas(apostas);

    if (verbosityLevel > 1)
    System.out.println(NLSHelper.getMessage("facade.persisting_ranking"));
    // Persistir ranking
    for (int i = 0; i < rankedApostas.size(); i++) {
        Aposta rankedAposta = rankedApostas.get(i);
        apostaDAO.insertRankingAposta(rankedAposta.getId(),
        rankedAposta.getPontuacaoTotal(), i + 1);
    }

    if (verbosityLevel > 0) System.out.println(NLSHelper.getMessage("facade.top_10_bets"));
    int limit = Math.min(10, rankedApostas.size());
    for (int i = 0; i < limit; i++) {
        System.out.println(NLSHelper.getMessage("facade.rank_entry") + (i + 1) + ": " +
        rankedApostas.get(i).toString());
    }

    // Análise de desempenho contra o último sorteio (para tabela FAT_APOSTAS)
    if (verbosityLevel > 0)
    System.out.println(NLSHelper.getMessage("facade.analyzing_performance"));
    analyzePerformance(rankedApostas);
}

/**
 * Analisa o desempenho das apostas geradas em relação ao último sorteio e persiste em
 * FAT_APOSTAS.
 * @param apostasGeradas Lista das apostas geradas e ranqueadas.
 * @throws SQLException
 */
private void analyzePerformance(List<Aposta> apostasGeradas) throws SQLException {
    Sorteio ultimoSorteio = statsDAO.getUltimoSorteio();
    if (ultimoSorteio == null) {
        System.out.println(NLSHelper.getMessage("facade.no_last_draw_found"));
        return;
    }

    if (verbosityLevel > 1) System.out.println(NLSHelper.getMessage("facade.last_draw_details")
    + ultimoSorteio.getConcurso() + " - " + ultimoSorteio.getDezenasSorteadas());

```

```

// Simular a "pontuação real" (número de acertos)
List<Aposta> apostasComAcertos = new ArrayList<>();
for (Aposta aposta : apostasGeradas) {
    int acertos = 0;
    for (int dezenaAposta : aposta.getDezenas()) {
        if (ultimoSorteio.getDezenasSorteadas().contains(dezenaAposta)) {
            acertos++;
        }
    }
    // A pontuação real pode ser o número de acertos diretamente ou uma escala
    aposta.setPontuacaoTotal(acertos); // Reutiliza pontuacaoTotal para o contexto de acertos
    reais
    apostasComAcertos.add(aposta);
}

// Ranqueia novamente com base nos acertos reais
apostasComAcertos.sort(Collections.reverseOrder()); // Re-ranqueia por acertos

if (verbosityLevel > 1)
System.out.println(NLSHelper.getMessage("facade.persisting_fat_bets"));
for (int i = 0; i < apostasComAcertos.size(); i++) {
    Aposta aposta = apostasComAcertos.get(i);
    FatAposta fatAposta = new FatAposta(
        aposta.getId(),
        LocalDate.now(), // Data da aposta
        ultimoSorteio.getDataSorteio(),
        ultimoSorteio.getConcurso(),
        aposta.getPontuacaoTotal(), // Soma do ranking de estratégias (do ranqueamento
        anterior)
        new ArrayList<>(ultimoSorteio.getDezenasSorteadas()),
        aposta.getPontuacaoTotal(), // Pontuação real (acertos)
        i + 1 // Ranking real
    );
    apostaDAO.insertFatAposta(fatAposta);
}
if (verbosityLevel > 0)
System.out.println(NLSHelper.getMessage("facade.performance_analysis_complete"));
}
}

```

- **NLSHelper (Novas Chaves no messages.properties e messages_pt_BR.properties):**
Properties


```
# ... (existentes) ...
facade.generating_random_bets=Generating {0} random bets...
facade.generating_lexico_bet=Generating lexicographical bet at index {0}...
facade.lexico_index_error=Lexicographical index error: {0}
facade.evaluating_full_combinations=Evaluating all 50,063,860 combinations (streaming mode)...
facade.reading_bets_from_file=Reading bets from file: {0}...
facade.invalid_line=Invalid line in file (not 6 numbers separated by comma): {0}
facade.invalid_number_in_line=Invalid number in line: {0}
facade.invalid_bet_in_line=Invalid bet in line (e.g., duplicate or out of range): {0}
facade.no_bets_to_process=No bets to process.
facade.evaluating_bets=Evaluating bets with strategies...
facade.processed_bets_count=Processed {0} bets.
facade.error_processing_bet=Error processing a bet: {0}
facade.ranking_bets=Ranking bets...
facade.persisting_ranking=Persisting ranking...
facade.top_10_bets=Top 10 Bets:
facade.rank_entry=Rank {0}: {1}
facade.analyzing_performance=Analyzing performance against the last draw...
facade.no_last_draw_found=No last draw found in the database. Performance analysis skipped.
facade.last_draw_details=Last Draw (Concurso {0}): {1}
facade.persisting_fat_bets=Persisting performance analysis in FAT_APOSTAS...
facade.performance_analysis_complete=Performance analysis complete.
```

(e suas traduções correspondentes em messages_pt_BR.properties)

5. Considerações Finais sobre a Implementação e Próximos Passos

- **Geração Lexicográfica Completa:** A implementação de `apostaGenerator.generateLexicoAposta(long index)` é um esboço. A lógica real para calcular a k-ésima combinação lexicográfica é matematicamente intensiva e fora do escopo de um exemplo conciso aqui, mas a arquitetura está pronta para recebê-la. O `generateFullCombinations()` também é simulado; em um sistema real, ele seria um iterador que não materializa todas as combinações em memória.
- **Verbosity Level:** A classe `MegaSenaFacade` utiliza o `verbosityLevel` para controlar a quantidade de informações impressas no console.
- **Tratamento de Erros e Robustez:** O `RetryMechanism` e os blocos `try-catch` nos DAOs e no `Facade` são cruciais para a robustez da aplicação.
- **Métricas de Desempenho:** A tabela `FAT_APOSTAS` foi implementada para armazenar métricas de desempenho das apostas geradas contra resultados reais, permitindo análises futuras.

Próximo e Último Grande Passo: Geração dos Artefatos Finais

O próximo e último passo significativo será a demonstração da compilação, a geração do

FATJAR e a descrição de como executar a aplicação. Incluiremos também os arquivos de documentação finais.